Nithin S
221IT085

IT468 - Quantum Computing Lab Assignment 3

Colab Link : QC_3.ipynb

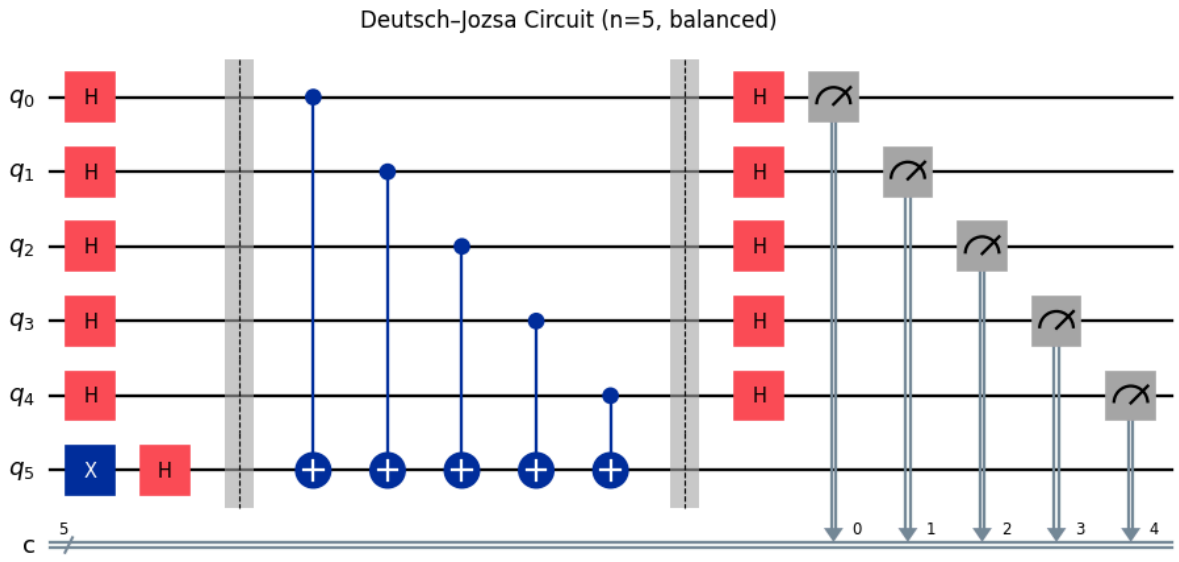
1. Implement the Deutsch-Jozsa algorithm for multiple Qubits using Deutsch's algorithm concept

Balanced

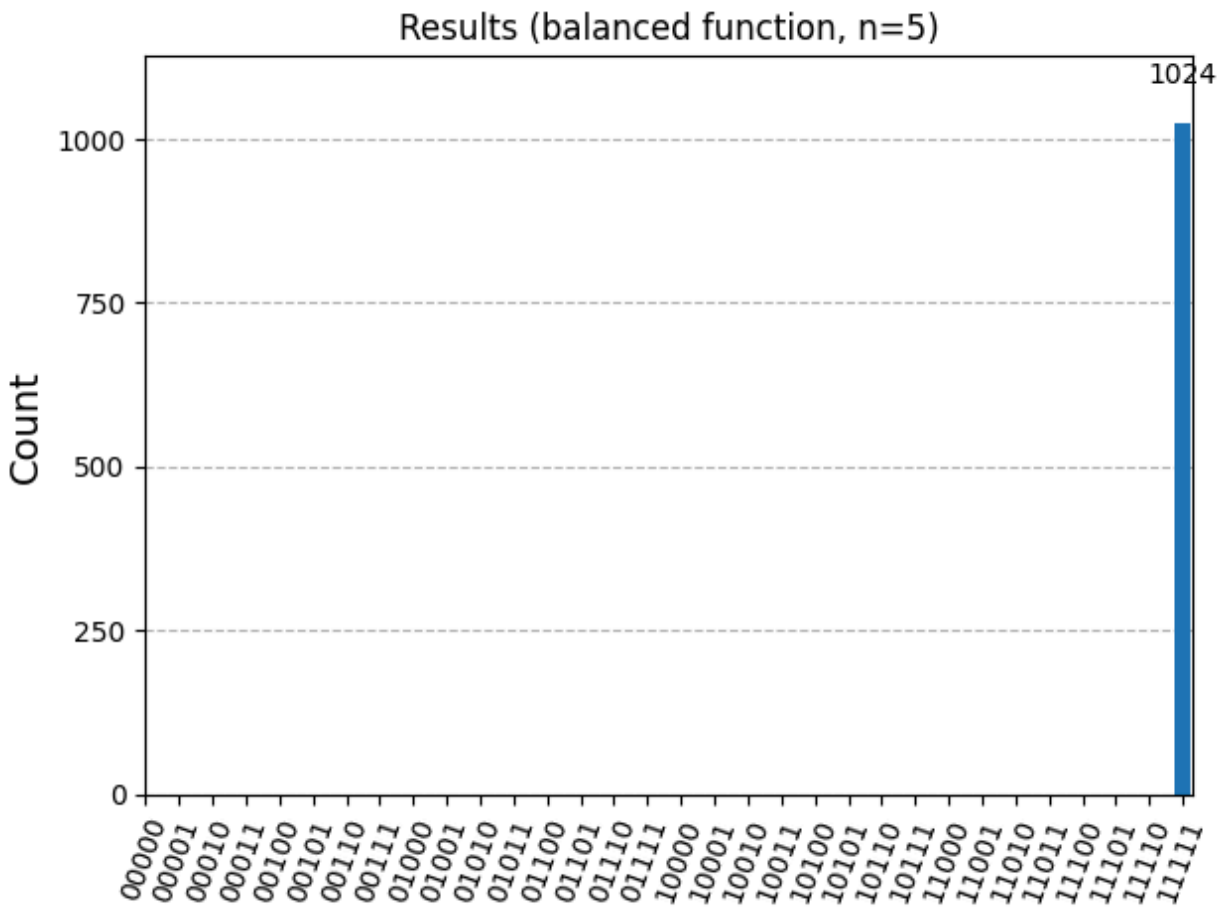
```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
from itertools import product
%matplotlib inline


def create_dj_oracle(n: int, case: str = "balanced") -> QuantumCircuit:
    oracle_qc = QuantumCircuit(n + 1, name=f"Oracle ({case})")
    if case == "balanced":
        for i in range(n):
            oracle_qc.cx(i, n)
    elif case == "constant":
        pass
    else:
        raise ValueError("case must be 'balanced' or 'constant'")
    return oracle_qc


def create_deutsch_jozsa_circuit(n: int, case: str = "balanced") ->
QuantumCircuit:
    qc = QuantumCircuit(n + 1, n)
    qc.h(range(n))
    qc.x(n)
    qc.h(n)
```

```
    qc.barrier()
    oracle = create_dj_oracle(n, case)
    qc = qc.compose(oracle)
    qc.barrier()
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc


n = 5
function_type = "balanced"

dj_circuit = create_deutsch_jozsa_circuit(n, function_type)
print(f"--- Circuit for n={n} and {function_type} function ---")

fig, ax = plt.subplots(figsize=(12, 6))
dj_circuit.draw(output="mpl", ax=ax, style="iqp")
plt.title(f"Deutsch-Jozsa Circuit (n={n}, {function_type})")
plt.show()

simulator = AerSimulator()
transpiled_circuit = transpile(dj_circuit, simulator)
job = simulator.run(transpiled_circuit, shots=1024)
result = job.result()
counts = result.get_counts(transpiled_circuit)

print(f"\n--- Results ---")
print(f" > Measurement counts: {counts}")

most_likely_outcome = max(counts, key=counts.get)
print("\n--- Decision ---")
if all(bit == '0' for bit in most_likely_outcome):
    print(f"Result '{most_likely_outcome}' is zero vector. Function is
constant.")
else:
    print(f"Result '{most_likely_outcome}' is nonzero. Function is
balanced.")

all_outcomes = [''.join(bits) for bits in product('01', repeat=n)]
for outcome in all_outcomes:
```

```
    counts.setdefault(outcome, 0)

hist = plot_histogram(counts, title=f"Results ({function_type} function,
n={n})", sort='asc')
```




```
--- Results ---
 > Measurement counts: {'11111': 1024}

--- Decision ---
Result '11111' is nonzero. Function is balanced.
```

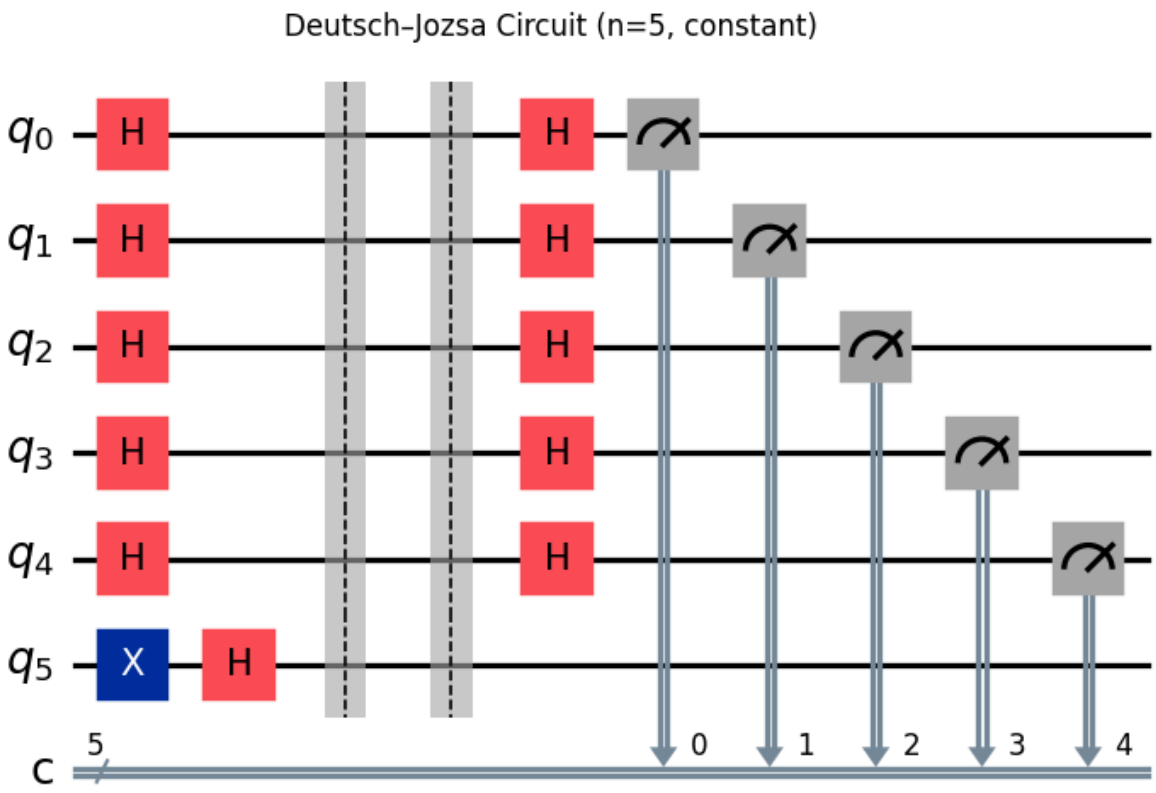


Constant

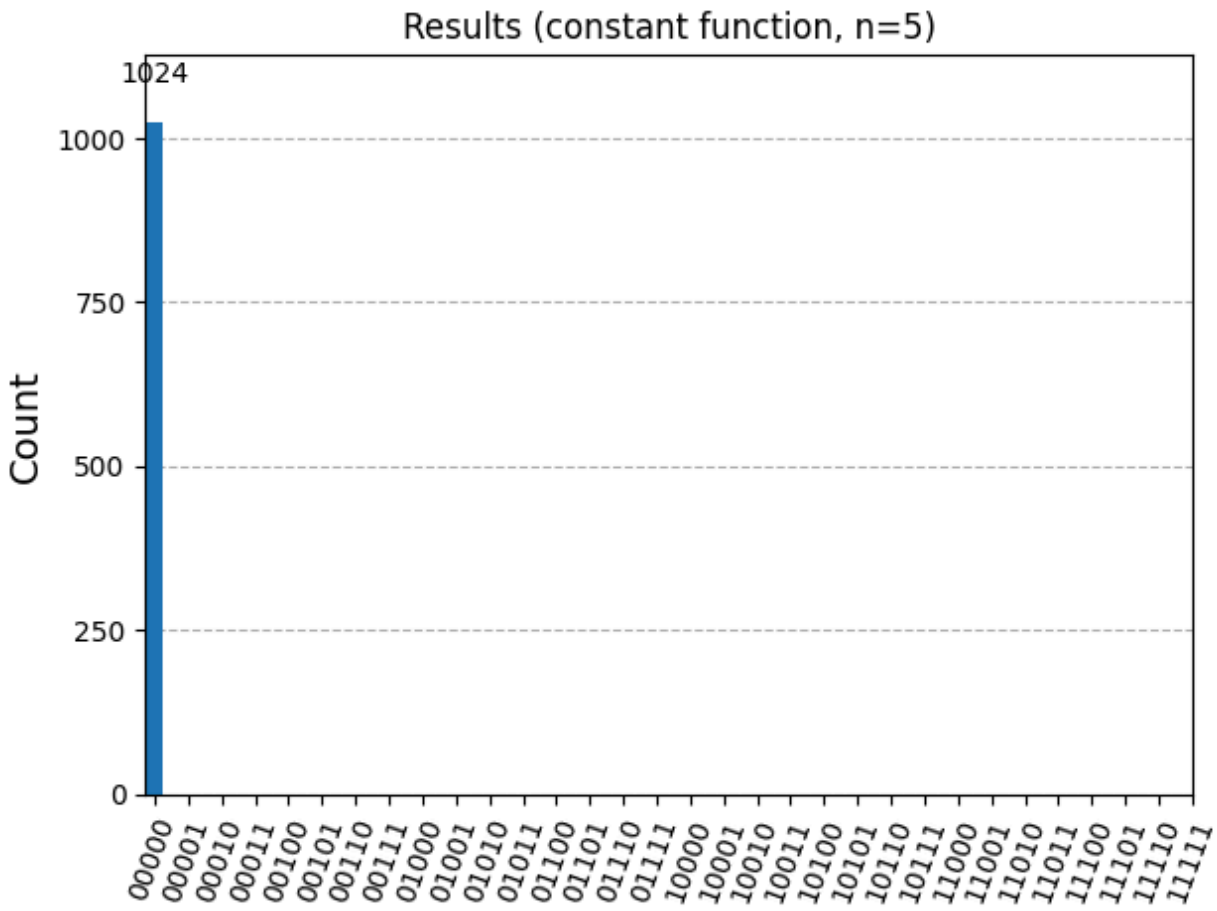
```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
from itertools import product
%matplotlib inline


def create_dj_oracle(n: int, case: str = "balanced") -> QuantumCircuit:
    oracle_qc = QuantumCircuit(n + 1, name=f"Oracle ({case})")
    if case == "balanced":
        for i in range(n):
            oracle_qc.cx(i, n)
    elif case == "constant":
```

```
        pass
    else:
        raise ValueError("case must be 'balanced' or 'constant'")
    return oracle_qc


def create_deutsch_jozsa_circuit(n: int, case: str = "balanced") ->
QuantumCircuit:
    qc = QuantumCircuit(n + 1, n)
    qc.h(range(n))
    qc.x(n)
    qc.h(n)
    qc.barrier()
    oracle = create_dj_oracle(n, case)
    qc = qc.compose(oracle)
    qc.barrier()
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc


n = 5
function_type = "constant"

dj_circuit = create_deutsch_jozsa_circuit(n, function_type)
print(f"--- Circuit for n={n} and {function_type} function ---")

fig, ax = plt.subplots(figsize=(12, 6))
dj_circuit.draw(output="mpl", ax=ax, style="iqp")
plt.title(f"Deutsch-Jozsa Circuit (n={n}, {function_type})")
plt.show()

simulator = AerSimulator()
transpiled_circuit = transpile(dj_circuit, simulator)
job = simulator.run(transpiled_circuit, shots=1024)
result = job.result()
counts = result.get_counts(transpiled_circuit)

print(f"\n--- Results ---")
print(f" > Measurement counts: {counts}")
```

```
most_likely_outcome = max(counts, key=counts.get)
print("\n--- Decision ---")
if all(bit == '0' for bit in most_likely_outcome):
   print(f"Result '{most_likely_outcome}' is zero vector. Function is
constant.")
else:
   print(f"Result '{most_likely_outcome}' is nonzero. Function is
balanced.")

all_outcomes = [''.join(bits) for bits in product('01', repeat=n)]
for outcome in all_outcomes:
   counts.setdefault(outcome, 0)

hist = plot_histogram(counts, title=f"Results ({function_type} function,
n={n})", sort='asc')
```



```
--- Results ---
 > Measurement counts: {'00000': 1024}

--- Decision ---
Result '00000' is zero vector. Function is constant.
```



We use simple notation: `|x⟩` is an n-qubit input state, `|y⟩` is the ancilla qubit. All qubits start as `|0...0⟩ |0⟩`. Flip the ancilla to `|1⟩` then apply Hadamard: `|1⟩ → H → (|0⟩ - |1⟩)/√2 = |-⟩`. So the initial state becomes `|0...0⟩ ⊗ |-⟩`.

Apply Hadamard to all input qubits (first H layer): `H^⊗n |0...0⟩ = 1/√(2^n) ∑_{x ∈ {0,1}^n} |x⟩`. Combined with ancilla: `1/√(2^n) ∑_x |x⟩ ⊗ |-⟩`.

Oracle (ancilla form) maps: `|x⟩|y⟩ → |x⟩ |y ⊕ f(x)⟩`. Since the ancilla is `|-⟩ = (|0⟩ - |1⟩)/√2`, flipping y when f(x)=1 adds a phase: `|-⟩ → (-1)^{f(x)} |-⟩`. So after the oracle, the state becomes `1/√(2^n) ∑_x (-1)^{f(x)} |x⟩ ⊗ |-⟩`. Note: the ancilla

remains |−⟩, and all information about the function is encoded as a phase factor attached to each |x⟩.

Second Hadamard layer on inputs: $H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_z (-1)^{x \cdot z} |z\rangle$. The amplitude of the all-zero output $|0...0\rangle$ becomes $\frac{1}{2^n} \sum_x (-1)^{f(x)}$. Thus, the probability to measure $|0...0\rangle$ depends only on the sum of $(-1)^{f(x)}$ over all x.

Constant vs Balanced:

- Constant function (f(x)=0 for all x): $(-1)^{f(x)} = 1 \ \forall \ x$. Sum = 2^n → amplitude for $|0...0\rangle$ is maximal → measurement gives $000...0$.
- Balanced function (half 0s, half 1s): Half the terms = +1, half = -1 → sum = 0. Amplitude for $|0...0\rangle$ = 0 → you never measure $000...0$, result is some nonzero bitstring.

The ancilla-oracle method encodes f(x) as a phase, and the second Hadamard layer reveals whether the function is constant or balanced using interference.

We're running the **Deutsch–Jozsa algorithm twice**:

- once with the **constant oracle**
- once with the **balanced oracle**
  and then comparing results.

2. Constant Oracle: The oracle ignores inputs and always gives the same output (either always 0 or always 1). After the algorithm finishes, the measurement result is always the **zero vector** (00000 for 5 qubits). This tells us the function is **constant**.

3. Balanced Oracle: The oracle flips the output depending on the input qubits, so exactly half inputs give 0 and half give 1. After the algorithm finishes, the measurement is **never the zero vector**. Instead, you get a nonzero bitstring (like 11111 or something similar). This tells us the function is **balanced**.

For **constant**: Histogram will show only 00000.
For **balanced**: Histogram will show other outcomes (not all zero).

2. Explain the Hadamard tensor product using an example of two matrices A and B of the same dimension $m \times n$, the Hadamard product $A \odot B$ (sometimes $A \circ B$)

$$\begin{bmatrix} 2 & 3 & 1 \\ 0 & 8 & -2 \end{bmatrix} \circ \begin{bmatrix} 3 & 1 & 4 \\ 7 & 9 & 5 \end{bmatrix} = ?$$

The **Hadamard product** (also called the **Hadamard tensor product** or **element-wise multiplication**) is just multiplying matrices **entry by entry**, instead of the usual row-by-column matrix multiplication.

Definition

If you have two matrices A and B of the **same size** (m×n), their Hadamard product is written as:

$$A \circ B = [a_{ij} \cdot b_{ij}]$$

That means: take the element in row iii, column jjj from A and multiply it with the element in the **same position** from B.